

Deployment Dokumentation

tipp app WM Tippspiel 2026

Inhaltsverzeichnis

1. Einleitung	4
Voraussetzungen	4
2. Vorbereitung des Repositories	4
2.1 Branch sauber halten	4
2.2 Repository auf GitHub publizieren	4
2.3 Umgebungsvariablen prüfen	4
2.4 Secrets erzeugen	5
3. Datenbank aufsetzen	5
3.1 Neon Projekt erstellen	5
3.2 Alternative Supabase	5
3.3 Lokale Verbindung prüfen	5
4. Migration und Seed der Produktionsdatenbank	5
4.1 Schema synchronisieren	5
4.2 Seed ausführen	6
4.3 Verifikation	6
5. Vercel Projekt anlegen	6
5.1 Account verbinden	6
5.2 Framework Erkennung	6
5.3 Build Befehl	6
6. Environment Variables in Vercel	6
7. Erstes Deployment auslösen	7
7.1 Logs prüfen	7
7.2 Typische Fehlerbilder	7
8. Custom Domain und PWA Verifikation	7
8.1 Eigene Domain verbinden	7
8.2 PWA Prüfung	7
9. Admin User anlegen	8
9.1 Variante via Prisma Studio	8
9.2 Variante via SQL	8
10. Smoke Test	8
11. Wartung im laufenden Turnier	8
11.1 Resultate erfassen	8
11.2 K.o. Paarungen pflegen	9

11.3 Updates ausrollen	9
11.4 Datenbank Backups	9
12. Anhang	9
12.1 Wichtige Pfade im Code	9
12.2 Nuetzliche Befehle	9
12.3 Checkliste vor Go Live	9

1. Einleitung

Diese Dokumentation beschreibt den vollständigen Weg vom aktuellen Stand der Codebasis bis zur produktiv erreichbaren Web Anwendung. Die App tipp app ist eine Progressive Web App für das Tippspiel zur Fussball Weltmeisterschaft 2026. Sie basiert auf Next.js 16, React 19, Prisma 6 mit PostgreSQL, Tailwind CSS 4 sowie einer JWT basierten Authentifizierung via jose.

Empfohlener Hosting Stack: Vercel für die Web Anwendung und Neon als verwaltete PostgreSQL Datenbank. Beide Anbieter bieten einen kostenlosen Tarif, integrieren sich nahtlos mit Next.js sowie Prisma und sind in Minuten betriebsbereit. Alternativ funktioniert Supabase mit identischem Vorgehen. Wo es Unterschiede gibt, ist dies vermerkt.

Voraussetzungen

- Ein GitHub Konto sowie installiertes Git auf dem Arbeitsplatz
- Node.js 20 oder neuer (lokal für Migration und Seed)
- Konto bei Vercel und Neon (oder Supabase)
- Funktionierender Build im lokalen Repository (npm run build)

2. Vorbereitung des Repositories

Vor dem ersten Deployment muss die Codebasis auf einem für Vercel erreichbaren Git Anbieter liegen. Im Folgenden werden die Schritte für GitHub beschrieben.

2.1 Branch sauber halten

Prüfe, dass der Branch main einen sauberen Status besitzt und alle Änderungen committed sind.

```
git status
git checkout main
git pull origin main
```

2.2 Repository auf GitHub publizieren

Lege auf GitHub ein neues, leeres Repository an, idealerweise mit dem Namen tipp app. Anschliessend lokal die Remote setzen und den Code pushen.

```
git remote add origin git@github.com:<dein-user>/tipp-app.git
git branch -M main
git push -u origin main
```

Hinweis. Falls SSH noch nicht eingerichtet ist, ist HTTPS ebenso möglich. Die URL lautet dann `https://github.com/<dein-user>/tipp-app.git` und GitHub fragt nach einem Personal Access Token.

2.3 Umgebungsvariablen prüfen

Die App benötigt zwei Variablen. Die Vorlage liegt unter `.env.example` und muss in der Produktion zwingend mit echten Werten gesetzt werden.

```
DATABASE_URL="postgresql://user:passwort@host:5432/tippspiel"
AUTH_SECRET="hier-ersetzen"
```

Achtung. Der Variablenname lautet `AUTH_SECRET` (nicht `JWT_SECRET`). Er wird in `src/lib/auth.ts` gelesen und dient als Schlüssel für die Signatur der Session Cookies. Ohne gesetzte Variable greift ein unsicherer Standardwert, der nur für die lokale Entwicklung gedacht ist.

2.4 Secrets erzeugen

Generiere ein kryptografisch sicheres Geheimnis. Die folgenden Befehle erzeugen jeweils einen Wert mit 256 Bit Entropie.

```
# macOS oder Linux
openssl rand -base64 32
```

```
# Alternativ via Node
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```

Hinweis. Speichere das erzeugte Geheimnis vorerst nur in einem Passwort Manager. In Vercel wird es später als Environment Variable hinterlegt und ist danach nicht mehr im Klartext einsehbar.

3. Datenbank aufsetzen

3.1 Neon Projekt erstellen

Registriere dich auf neon.tech und erstelle ein neues Projekt. Wähle als Region eine Lokation in der Nähe der Vercel Region (zum Beispiel Frankfurt). Neon liefert direkt einen Connection String.

- Im Dashboard auf **New Project** klicken
- Region wählen (Empfehlung: Europe central)
- Project Name: *tipp-app*
- Connection String aus dem Reiter **Connection Details** kopieren

Hinweis. Der von Neon angebotene *Pooled* Connection String (Port 5432, Host endet auf *-pooler*) ist für serverlose Umgebungen wie Vercel ideal. Für das Ausführen von Migrationen lokal empfiehlt sich der direkte Endpunkt ohne Pooler.

3.2 Alternative Supabase

Wer Supabase bevorzugt, erstellt unter supabase.com ein neues Projekt und kopiert unter **Project Settings > Database** den Connection String für die Anwendung. Das weitere Vorgehen ist identisch.

3.3 Lokale Verbindung prüfen

Trage den Connection String temporaer in deine lokale `.env` Datei ein und teste die Verbindung.

```
# Datei .env
DATABASE_URL="postgresql://...neon.tech/neondb?sslmode=require"
AUTH_SECRET="<dein-generiertes-secret>"

npx prisma db push --skip-generate
```

Der Befehl synchronisiert das Schema aus `prisma/schema.prisma` mit der Produktionsdatenbank. Da noch keine Migrationsverzeichnisse vorliegen, ist `db push` der einfachste Weg.

4. Migration und Seed der Produktionsdatenbank

Das Repository enthaelt ein Seed Skript unter `prisma/seed.ts`. Es befüllt die Datenbank mit den 48 Mannschaften, allen Gruppenspielen sowie Platzhaltern für die K.o. Phase.

4.1 Schema synchronisieren

```
npm install
npx prisma db push
```

4.2 Seed ausführen

```
npm run db:seed
```

Das Skript erzeugt die Teams, ordnet sie den Gruppen A bis L zu und legt die geplanten Begegnungen an. Bestehende Datensätze werden nach dem Prinzip upsert aktualisiert.

Hinweis. Der Befehl `npm run db:setup` kombiniert push und seed in einem Schritt und ist für das erstmalige Aufsetzen der Produktionsdatenbank gleichwertig nutzbar.

4.3 Verifikation

Prüfe im Anschluss, dass die Tabellen befüllt sind.

```
npx prisma studio
```

Prisma Studio öffnet eine lokale Web Oberfläche auf `http://localhost:5555` und visualisiert die Inhalte der Produktionsdatenbank.

5. Vercel Projekt anlegen

5.1 Account verbinden

- `vercel.com` aufrufen und mit dem GitHub Account anmelden
- Im Dashboard auf **Add New > Project** klicken
- Das Repository `tipp app` aus der Liste der GitHub Repositories importieren

5.2 Framework Erkennung

Vercel erkennt automatisch Next.js. Die Voreinstellungen können übernommen werden, mit einer wichtigen Anpassung beim Build Befehl.

5.3 Build Befehl

Stelle sicher, dass im Schritt **Build and Output Settings** der Build Befehl wie folgt lautet. Damit erzeugt Prisma während des Builds den Client basierend auf der aktuellen `schema.prisma`.

```
prisma generate &amp; next build
```

Hinweis. Dieser Befehl ist bereits im `package.json` als npm script build hinterlegt. Vercel ruft per Default `npm run build` auf, sodass keine manuelle Anpassung nötig ist. Bei Abweichung kann die obige Zeile explizit eingetragen werden.

6. Environment Variables in Vercel

Im Projekt unter **Settings > Environment Variables** werden die folgenden Einträge hinterlegt. Der Scope ist auf Production, Preview und Development zu setzen, damit alle Vercel Umgebungen funktionieren.

Schlüssel	Wert	Bemerkung
DATABASE_URL	postgresql://...neon.tech/...?sslmode=require	Connection String von Neon
AUTH_SECRET	Ergebnis von <code>openssl rand base64 32</code>	Signatur der Session Cookies

Achtung. Werte mit Sonderzeichen (insbesondere @, : und /) im Passwort des Connection Strings müssen URL kodiert werden. Andernfalls schlägt die Verbindung mit einer wenig sprechenden Fehlermeldung fehl.

7. Erstes Deployment auslösen

Nach dem Hinterlegen der Variablen kann das Deployment via **Deploy** Button gestartet werden. Vercel installiert die Abhängigkeiten, ruft prisma generate auf, baut die Next.js Anwendung und veröffentlicht sie unter einer vercel.app Subdomain.

7.1 Logs prüfen

- Im Vercel Dashboard unter **Deployments** den laufenden Build öffnen
- Phase **Building** auf Erfolg prüfen (next build muss ohne Fehler beenden)
- Phase **Deploying** liefert die finale URL
- Phase **Runtime Logs** nutzt man bei späteren Laufzeit Fehlern

7.2 Typische Fehlerbilder

Falls der Build oder die Laufzeit fehlschlägt, liefern die Logs häufig folgende Hinweise.

- **P1001:** Datenbank nicht erreichbar. DATABASE_URL prüfen, sslmode=require setzen.
- **P1003 / P3009:** Schema nicht vorhanden. Schritt 4.1 erneut ausführen.
- **Module not found:** Lockfile in Git fehlt. package-lock.json muss im Repository liegen.
- **500 mit Session Fehler:** AUTH_SECRET wurde nicht gesetzt oder weicht zwischen Umgebungen ab.

8. Custom Domain und PWA Verifikation

8.1 Eigene Domain verbinden

- Im Vercel Projekt unter **Settings > Domains** die gewünschte Domain eintragen
- Bei der DNS Stelle die von Vercel angezeigten A bzw. CNAME Einträge setzen
- Auf gültiges TLS Zertifikat warten (in der Regel wenige Minuten)

8.2 PWA Prüfung

Die App stellt unter public ein Manifest und Icons bereit. Nach dem Deployment ist Folgendes zu prüfen.

- Im Browser den Reiter **Application** der DevTools öffnen
- Manifest wird ohne Warnungen geladen
- Service Worker (sofern aktiviert) ist registriert
- Die Installationsoption **App installieren** erscheint im Adressfeld

Hinweis. Auf iOS Geräten kann die App nur via Safari über das Teilen Menue und den Eintrag **Zum Home Bildschirm** installiert werden.

9. Admin User anlegen

Die App kennt das Flag `isAdmin` am User Modell. Ein Admin Benutzer hat Zugriff auf erweiterte Funktionen wie das Erfassen von Resultaten. Da der Endpunkt für die Registration keinen Admin Status vergibt, wird das Flag direkt in der Datenbank gesetzt.

9.1 Variante via Prisma Studio

```
DATABASE_URL="..." npx prisma studio
```

Den gewünschten User in der Tabelle **User** öffnen, das Feld `isAdmin` auf `true` setzen und speichern.

9.2 Variante via SQL

Direkt in der Neon Konsole oder via `psql`.

```
UPDATE "User" SET "isAdmin" = true WHERE email = 'patrick@example.ch';
```

Achtung. Nach Änderung des `isAdmin` Flags ist eine erneute Anmeldung erforderlich, damit das aktualisierte Recht in der Session JWT enthalten ist.

10. Smoke Test

Bevor die App produktiv genutzt wird, sollte ein kompletter Durchlauf mit Testdaten erfolgen. Folgende Prüfliste deckt die wichtigsten Funktionen ab.

1. Aufruf der produktiven URL, Startseite laedt ohne Fehler
2. Registration eines neuen Testkontos via `/register`
3. Login mit den eben erstellten Daten via `/login`
4. Tipp für ein anstehendes Gruppenspiel abgeben und speichern
5. Tipp anschliessend bearbeiten und erneut speichern
6. Erstellen einer Tippgruppe und Erzeugen eines Invite Codes
7. Beitritt via Invite Code mit einem zweiten Testkonto
8. Rangliste öffnen und Sortierung prüfen
9. Logout und erneuter Login prüfen

Hinweis. Test User können nach dem Smoke Test entweder via Prisma Studio gelöscht oder behalten werden. Da kein automatischer Reset existiert, ist die manuelle Bereinigung die einfachste Lösung.

11. Wartung im laufenden Turnier

11.1 Resultate erfassen

Sobald ein Spiel beendet ist, werden `homeScore`, `awayScore` sowie der Status auf `FINISHED` gesetzt. Solange kein Admin Interface gebaut ist, erfolgt dies via Prisma Studio oder SQL.

```
UPDATE "Match"  
SET "homeScore" = 2, "awayScore" = 1, "status" = 'FINISHED'  
WHERE id = 12;
```

Die Punkte werden beim Aufruf der Rangliste anhand der Logik in `src/lib/scoring.ts` berechnet und in `Tip.points` gespeichert.

11.2 K.o. Paarungen pflegen

Die K.o. Matches enthalten in der Datenbank Platzhalter in homePlaceholder und awayPlaceholder (zum Beispiel 1A für den Gruppensieger A). Sobald die effektiven Mannschaften feststehen, werden homeTeamId und awayTeamId gesetzt.

```
UPDATE "Match"  
SET "homeTeamId" = 'SUI', "awayTeamId" = 'BRA'  
WHERE id = 73;
```

11.3 Updates ausrollen

Jeder Push auf den Branch main löst in Vercel automatisch ein neues Deployment aus. Vorher empfiehlt sich lokal ein Smoke Build.

```
npm run lint  
npm run build
```

11.4 Datenbank Backups

Neon erstellt im Free Tarif automatische Branches und Snapshots der Daten. Für das Tippspiel reicht das in der Regel. Bei kritischen Änderungen empfiehlt sich vor dem Eingriff ein manueller pg_dump.

```
pg_dump "$DATABASE_URL" --no-owner --no-acl &gt; backup.sql
```

12. Anhang

12.1 Wichtige Pfade im Code

- **prisma/schema.prisma** Datenmodell der App
- **prisma/seed.ts** Initialdaten für Teams und Spiele
- **src/lib/auth.ts** Sessions und AUTH_SECRET
- **src/lib/db.ts** Prisma Client Singleton
- **src/lib/scoring.ts** Berechnung der Tipppunkte
- **src/app** Next.js App Router Routen

12.2 Nützliche Befehle

```
# Entwicklung starten  
npm run dev  
  
# Schema in DB pushen  
npm run db:push  
  
# Seed Daten laden  
npm run db:seed  
  
# Datenbank in Browser anzeigen  
npx prisma studio  
  
# Produktiver Build  
npm run build && npm start
```

12.3 Checkliste vor Go Live

- Repository auf GitHub aktuell

- Neon Projekt erstellt, Connection String notiert
- AUTH_SECRET generiert und sicher gespeichert
- prisma db push und prisma seed erfolgreich
- Vercel Projekt importiert, Build erfolgreich
- DATABASE_URL und AUTH_SECRET in Vercel gesetzt
- Smoke Test bestanden
- Admin User aktiviert
- Custom Domain konfiguriert (optional)

Damit ist die App startklar für das Tippspiel zur WM 2026. Viel Erfolg.